



รายงาน

การประยุกต์ใช้ Algorithm แบบเส้นทางเดียวในการแก้ปัญหาการหาค่าต่ำสุดสากล
กรณีศึกษาฟังก์ชัน Rastrigin

รายวิชา FRA502 Advanced Robotic Control and Simulation with ROS2 (ARCS2)

สถาบันวิทยาการหุ่นยนต์ภาคสนาม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ประจำปีการศึกษา 1/2568

ผู้จัดทำ

นายภคิน บุญชนะชัย 66340500037

บทนำ

ในการแก้ปัญหาการหาค่าเหมาะสมที่สุด (Optimization) ในงานวิจัยและงานประยุกต์ใช้งานจริง จุดประสงค์หลักคือการหาค่าที่ทำให้ฟังก์ชันวัตถุประสงค์ (Objective Function) มีค่าน้อยที่สุดหรือมากที่สุด ค่าต่ำสุดสากล (Global Minimum) หมายถึงค่าที่ต่ำที่สุดของ Function ในทั้งหมดของปริภูมิการค้นหา (Search Space)

อย่างไรก็ตาม ปัญหาสำคัญที่พบในการหาค่าเหมาะสมที่สุดคือการมีอยู่ของค่าต่ำสุดเฉพาะที่ (Local Minima) จำนวนมากใน Function ที่มีความซับซ้อนสูง ค่าต่ำสุดเฉพาะที่หมายถึงจุดที่มีค่าต่ำกว่าบริเวณใกล้เคียงโดยรอบ แต่ไม่ใช่ค่าต่ำที่สุดของทั้งระบบ อัลกอริทึมการหาค่าเหมาะสมที่สุดแบบดั้งเดิม เช่น Gradient Descent มักประสบปัญหาการติดอยู่ใน Local Minima และไม่สามารถค้นหาต่อไปยัง Global Minimum ที่แท้จริงได้

เปรียบเทียบกับการเดินทางในพื้นที่ภูมิประเทศที่มีหุบเขาหลายแห่ง หากเดินลงเนินโดยตรงจากจุดเริ่มต้น จะพบหุบเขาแห่งแรก (Local Minimum) และไม่สามารถเคลื่อนที่ต่อไปได้ เนื่องจากทุกทิศทางมีความสูงเพิ่มขึ้น แม้ว่าจะมีหุบเขาที่ลึกกว่า (Global Minimum) อยู่ในบริเวณอื่นของพื้นที่

Rastrigin Function และความท้าทายในการแก้ปัญหา

Rastrigin Function เป็น Function มาตรฐานที่ใช้ในการทดสอบประสิทธิภาพของ Algorithms การหาค่าเหมาะสมที่สุดแบบสากล (Global Optimization Algorithms) อย่างแพร่หลาย เนื่องจากมีลักษณะที่ท้าทายสูงด้วยการมี Local Minima จำนวนมาก

สมการคณิตศาสตร์ของ Rastrigin Function มีรูปแบบดังนี้:

$$f(x) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

โดยที่: $A = 10$ ค่าคงที่ Parameter
 n จำนวนมิติของปริภูมิการค้นหา (Number of Dimensions)
 $x_i \in [-5.12, 5.12]$ สำหรับทุก $i = 1, 2, \dots, n$

ค่าต่ำสุดสากล (Global Minimum) ของ Function นี้อยู่ที่จุดกำเนิด $x^* = (0, 0, \dots, 0)$ และมีค่า $f(x^*) = 0$

ในการศึกษานี้จะนำเสนอการวิเคราะห์และเปรียบเทียบ Algorithms การหาค่าเหมาะสมที่สุดแบบสากลสองวิธี ที่ใช้แนวทาง Single-Trajectory (การเดินทางเส้นทางเดียว) โดยไม่ใช้วิธีการ Population-based หรือการสร้างตัวแทนหลายตัวพร้อมกัน Algorithms ทั้งสองที่จะนำเสนอประกอบด้วย

1. Dual Annealing: Algorithms ที่รวมหลักการของ Generalized Simulated Annealing บนพื้นฐานของสถิติ Tsallis เข้ากับการค้นหาเฉพาะที่ (Local Search) เพื่อให้ได้ทั้งความสามารถในการสำรวจปริภูมิแบบกว้าง (Global Exploration) และการหาค่าเหมาะสมที่สุดแบบละเอียด (Local Exploitation) พร้อมกัน
2. Basin Hopping: Algorithms ที่ใช้กลไกการรบกวนแบบสุ่ม (Random Perturbation) ร่วมกับการหาค่าต่ำสุดเฉพาะที่ (Local Minimization) ซ้ำๆ และใช้เกณฑ์การยอมรับแบบ Metropolis เพื่อตัดสินใจเลือกสถานะใหม่

Dual Annealing Algorithms

ทฤษฎีและหลักการ

Dual Annealing เป็นวิธีการหาค่าเหมาะสมที่สุดแบบสากลที่พัฒนาจากการผสมผสาน Generalized Simulated Annealing (GSA) บนพื้นฐานสถิติ Tsallis กับการค้นหาเฉพาะที่ (Local Search) จุดเด่นของ Algorithms นี้คือการใช้ Distorted Cauchy-Lorentz Distribution ในการสร้างจุดใหม่ ซึ่งมีหางหนา (Fat tail) สามารถกระโดดไกลได้มากกว่า Gaussian Distribution ทำให้หนีจาก Local Minima ได้ง่ายขึ้น และมีกลไก Reannealing ที่ป้องกันการติดอยู่ในบริเวณเดิม

สมการคณิตศาสตร์ที่เกี่ยวข้อง

1. Visiting Distribution (การแจกแจงสำหรับการสร้างจุดใหม่)

$$g_{q_v}(\Delta x(t)) \propto \frac{[T_{q_v}(t)]^{-\frac{D}{3-q_v}}}{\left[1 + (q_v - 1) \frac{(\Delta x(t))^2}{[T_{q_v}(t)]^{\frac{2}{3-q_v}}}\right]^{\frac{1}{q_v-1} + \frac{D-1}{2}}}$$

โดยที่ $\Delta x(t)$ คือระยะการกระโดด, $T_{q_v}(t)$ คืออุณหภูมิ, D คือจำนวนมิติของปัญหา, q_v คือ Parameter การเยื่อมชมที่ควบคุมความหนาของหาง การแจกแจงแบบ Cauchy-Lorentz มีลักษณะ "หางหนา" (Fat-tailed) ทำให้มีความน่าจะเป็นสูงในการสร้างจุดที่ห่างไกลจากตำแหน่งปัจจุบัน เมื่อเทียบกับ Gaussian Distribution ที่มักสร้างจุดใกล้เคียงเท่านั้น ความสามารถในการกระโดดไกลนี้เป็นกลไกหลักในการหลีกเลี่ยง Local Minima

2. Acceptance Probability (ความน่าจะเป็นในการยอมรับ)

$$p_{q_a} = \min \left\{ 1, [1 - (1 - q_a)\beta\Delta E]^{-\frac{1}{1-q_a}} \right\}$$

โดยที่ q_a คือ Acceptance Parameter, $\beta = \frac{1}{T}$, $\Delta E = E_{\text{new}} - E_{\text{current}}$ คือผลต่างของค่าพลังงาน

เมื่ออุณหภูมิ T สูง (ช่วงเริ่มต้น) Algorithms มีแนวโน้มยอมรับจุดที่แย่กว่าได้ง่าย สนับสนุนการสำรวจพื้นที่กว้าง เมื่อ T ลดลงตามเวลา การยอมรับจะเข้มงวดขึ้นทำให้ Algorithms มุ่งเน้นการหาค่าเหมาะสมที่สุดในบริเวณที่มีศักยภาพ

3. Temperature Schedule (ตารางการลดอุณหภูมิ)

$$T_{q_v}(t) = T_{\text{initial}} \cdot t^{-\frac{1}{D-1}}$$

การลดอุณหภูมิแบบ power-law นี้เร็วกว่าการลดแบบ logarithmic ของ CSA แบบดั้งเดิม

ขั้นตอนการทำงาน หมายเหตุ: ใช้ L-BFGS-B (Gradient-Based Local Optimizer) สำหรับหาค่าต่ำสุดเฉพาะที่

1. **เริ่มต้น:** กำหนด x_0, T_0 และ Parameter q_v, q_a
2. **Visiting Phase:** สุ่ม Δx จาก Visiting Distribution และคำนวณ $x_{\text{trial}} = x_{\text{current}} + \Delta x$
3. **Local Minimization:** ใช้ L-BFGS-B หา Local Minimum จาก x_{trial} ได้ x_{local}
4. **Acceptance Test:** คำนวณ $\Delta E = f(x_{\text{local}}) - f(x_{\text{current}})$ และตัดสินใจยอมรับด้วย p_{q_a}
5. **Update:** อัปเดต x_{current} และ x_{best} ตามผลการยอมรับ
6. **Temperature Adjustment:** ลดอุณหภูมิตาม Schedule และตรวจสอบ Reannealing
7. **Termination:** ตรวจสอบเงื่อนไขหยุด ถ้ายังไม่ครบกลับไปขั้นตอน 2

Basin Hopping Algorithms

ทฤษฎีและหลักการ

Basin Hopping เป็นวิธีการหาค่าเหมาะสมที่สุดแบบสากลที่พัฒนาสำหรับหาโครงสร้างพลังงานต่ำสุดของ Cluster Atom แนวคิดหลักคือการแปลงพื้นผิวพลังงานที่ซับซ้อนให้เรียบขึ้นโดยทำ Local Minimization ทุกครั้งหลังจากเคลื่อนที่ไปยังตำแหน่งใหม่ อัลกอริทึมจะ "กระโดด" จาก Basin หนึ่งไปยังอีก Basin หนึ่งโดยเปรียบเทียบเฉพาะค่าต่ำสุดของแต่ละ Basin แทนที่จะเดินตามพื้นผิวที่มีขื่นๆ ลงๆ วิธีนี้ช่วยหลีกเลี่ยง Local Minima และใช้เกณฑ์ Metropolis ในการตัดสินใจยอมรับสถานะใหม่

สมการคณิตศาสตร์ที่เกี่ยวข้อง

1. Transformed Energy Landscape (การแปลงพื้นผิวพลังงาน)

$$\tilde{f}(x) = \min_{x' \in \text{basin}(x)} f(x')$$

Algorithms จะใช้ค่าต่ำสุดในบริเวณนั้น (Local Minimum) แทนที่จะใช้ค่า $f(x)$ โดยตรง แนวคิดหลักคือการแปลงพื้นผิวที่ซับซ้อนให้เรียบขึ้นโดยใช้เฉพาะค่า Local Minimum ของแต่ละ Basin แทนที่จะใช้ค่าจริงของ Function ทำให้ Algorithms "มองเห็น" เฉพาะความลึกของแต่ละหุบเขา ไม่สนใจความผันผวนรายละเอียดบนพื้นผิว

2. Random Perturbation (การรบกวนแบบสุ่ม)

$$x_{\text{trial}} = x_{\text{current}} + \Delta x, \quad \Delta x_i \sim \mathcal{U}(-s, +s)$$

โดยที่ s คือ Step Size ที่กำหนด และ $\mathcal{U}$ เป็นการสุ่มแบบ Uniform Distribution

3. Metropolis Acceptance Criterion (เกณฑ์การยอมรับแบบ Metropolis)

$$P_{\text{accept}} = \min \left\{ 1, \exp \left(-\frac{\Delta E}{k_B T} \right) \right\}$$

โดยที่ $\Delta E = f(x_{\text{local}}) - f(x_{\text{current}})$, k_B คือ Boltzmann constant (มักตั้งเป็น 1), T คือ อุณหภูมิ

กรณีพิเศษเมื่อ $T = 0$: Monotonic Basin Hopping

$$P_{\text{accept}} = \begin{cases} 1 & \text{if } \Delta E < 0 \\ 0 & \text{if } \Delta E \geq 0 \end{cases}$$

ขั้นตอนการทำงาน หมายเหตุ: ใช้ L-BFGS-B (Gradient-Based Local Optimizer) สำหรับหาค่าต่ำสุดเฉพาะที่

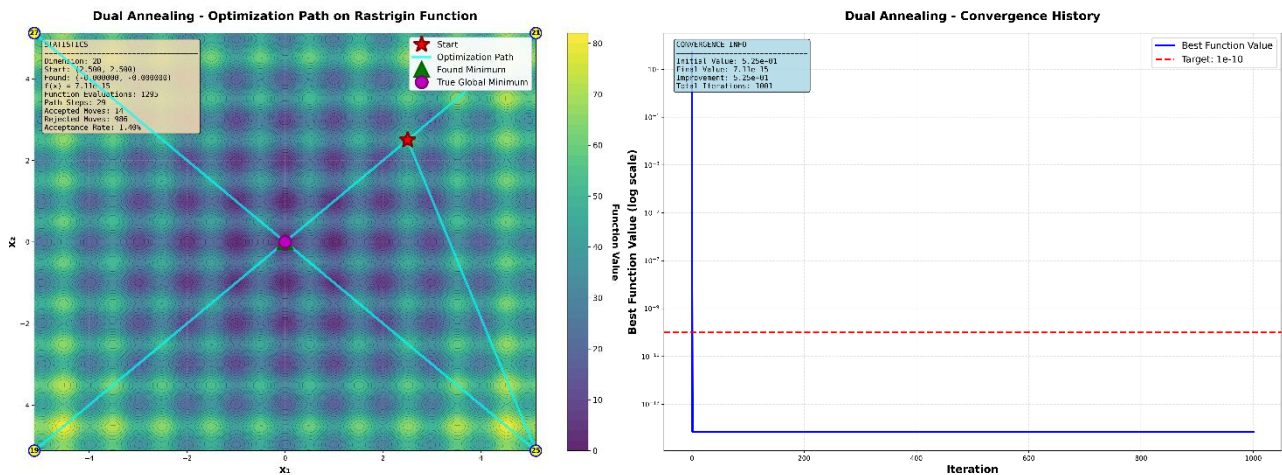
1. **เริ่มต้น:** กำหนด x_0 และทำ Local Minimization เพื่อได้ x_{current} (Local Minimum แรก)
2. **Perturbation Phase:** สุ่ม Δx จาก $\mathcal{U}(-s, +s)$ และคำนวณ $x_{\text{trial}} = x_{\text{current}} + \Delta x$
3. **Local Minimization:** ใช้ L-BFGS-B หา Local Minimum จาก x_{trial} ได้ x_{local}
4. **Acceptance Test:** คำนวณ ΔE และตัดสินใจยอมรับด้วย Metropolis Criterion
5. **Update:** อัปเดต x_{current} และ x_{best} ตามผลการยอมรับ
6. **Adaptive Stepsize:** ทุก I Iterations ปรับ stepsize เพื่อรักษา acceptance rate $\approx 50\%$
7. **Termination:** ตรวจสอบว่าครบ N Iterations หรือไม่ ถ้ายังไม่ครบกลับไปขั้นตอน 2

การทำการทดลอง

ตารางที่ 1: Parameter ที่ใช้ในการทดลอง Dual Annealing

พารามิเตอร์	สัญลักษณ์	ค่าที่ใช้	คำอธิบาย
initial_temp	T_0	5230.0	อุณหภูมิเริ่มต้นควบคุมความกว้างการสำรวจ
visit	q_v	2.62	พารามิเตอร์การเยี่ยมชม ควบคุมระยะกระโดด
accept	q_a	-5.0	พารามิเตอร์การยอมรับ ควบคุมความเข้มงวด
restart_temp_ratio	r	2×10^{-5}	อัตราส่วนการรีเซ็ตอุณหภูมิ
maxiter	N	1000	จำนวนรอบการค้นหาสูงสุด
minimizer_method	-	L-BFGS-B	วิธีการหาค่าต่ำสุดเฉพาะที่
bounds	-	[-5.12, 5.12]	ขอบเขตของตัวแปรในแต่ละมิติ
initial_point	x_0	[2.5, 2.5, ...]	จุดเริ่มต้นเดียวกันทุกมิติ

ผลลัพธ์จากการทดลอง Dual Annealing



รูปที่ 1

ตารางที่ 2: ผลการทดลอง Dual Annealing บน Rastrigin Function

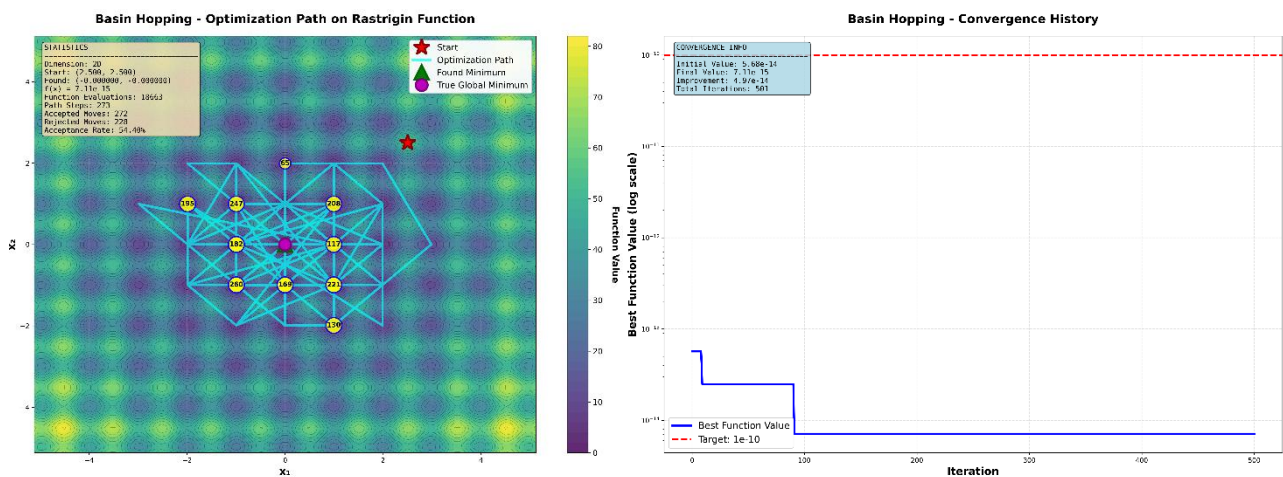
มิติ	จุดเริ่มต้น	ระยะห่างจาก Global Min	จำนวนการเรียกใช้ Function	Acceptance Rate
2D	[2.5, 2.5]	6.7×10^{-9}	1,295	1.40%
3D	[2.5, 2.5, 2.5]	8.5×10^{-9}	3,353	8.40%
4D	[2.5, 2.5, 2.5, 2.5]	1.03×10^{-8}	12,726	33.50%
5D	[2.5, 2.5, ..., 2.5]	7.4×10^{-9}	34,307	79.30%
6D	[2.5, 2.5, ..., 2.5]	1.20×10^{-8}	41,720	83.10%

หมายเหตุ: ค่าเริ่มต้นของฟังก์ชัน: 2D=52.5, 3D=78.75, 4D=105.0, 5D=131.25, 6D=157.5

ตารางที่ 3: Parameter ที่ใช้ในการทดลอง Basin Hopping

พารามิเตอร์	สัญลักษณ์	ค่าที่ใช้	คำอธิบาย
niter	N	500	จำนวนรอบ Basin Hopping iterations
T	T	2.0	พารามิเตอร์อุณหภูมิสำหรับการยอมรับ
stepsize	S	1.5	ขนาดก้าวสูงสุดสำหรับการรบกวนสุ่ม
interval	I	50	ช่วงการปรับ stepsize อัตโนมัติ
minimizer_method	-	L-BFGS-B	วิธีการหาค่าต่ำสุดเฉพาะที่
bounds	-	[-5.12, 5.12]	ขอบเขตของตัวแปรในแต่ละมิติ
initial_point	x_0	[2.5, 2.5, ...]	จุดเริ่มต้นเดียวกันทุกมิติ

ผลลัพธ์จากการทดลอง Basin Hopping



รูปที่ 2

ตารางที่ 4: ผลการทดลอง Basin Hopping บน Rastrigin Function

มิติ	จุดเริ่มต้น	ระยะห่างจาก Global Min	จำนวนการเรียกใช้ Function	Acceptance Rate
2D	[2.5, 2.5]	7.3×10^{-9}	18,663	54.40%
3D	[2.5, 2.5, 2.5]	2.14×10^{-8}	27,292	46.00%
4D	[2.5, 2.5, 2.5, 2.5]	2.59×10^{-8}	36,225	41.80%
5D	[2.5, 2.5, ...]	2.64×10^{-8}	44,058	41.40%
6D	[2.5, 2.5, ...]	3.10×10^{-8}	53,858	39.80%

หมายเหตุ: ค่าเริ่มต้นของฟังก์ชัน: 2D=52.5, 3D=78.75, 4D=105.0, 5D=131.25, 6D=157.5

การวิเคราะห์ผลลัพธ์

ตารางที่ 5: การเปรียบเทียบความสามารถระหว่างสอง Algorithms

มิติ	DA Function Calls	BH Function Calls	อัลกอริทึมที่ดีกว่า
2D	1,295	18,663	Dual Annealing (14× เร็วกว่า)
3D	3,353	27,292	Dual Annealing (8× เร็วกว่า)
4D	12,726	36,225	Dual Annealing (3× เร็วกว่า)
5D	34,307	44,058	Dual Annealing (1.3× เร็วกว่า)
6D	41,720	53,858	Dual Annealing (1.3× เร็วกว่า)

Dual Annealing และ Basin Hopping สามารถหา Global Minimum ของ Rastrigin Function ได้สำเร็จในทุกมิติ โดยค่าฟังก์ชันที่ได้มีค่าเป็นศูนย์หรือใกล้เคียงศูนย์มาก ซึ่งบ่งชี้ว่าทั้งสอง Algorithms สามารถเข้าสู่จุดกำเนิด ได้อย่างแม่นยำ ซึ่งสามารถวิเคราะห์ประสิทธิภาพระหว่าง 2 วิธีได้ดังนี้

ตารางที่ 6: การเปรียบเทียบสมรรถนะ

มิติ	DA Calls	BH Calls	อัตราส่วน (BH/DA)	DA Accept Rate	BH Accept Rate
2D	1,295	18,663	14.4	1.4%	54.4%
3D	3,353	27,292	8.1	8.4%	46.0%
4D	12,726	36,225	2.8	33.5%	41.8%
5D	34,307	44,058	1.3	79.3%	41.4%
6D	41,720	53,858	1.3	83.1%	39.8%

Dual Annealing แสดงประสิทธิภาพที่เหนือกว่าอย่างชัดเจนในทุกมิติ โดยใช้จำนวน Function Evaluations น้อยกว่า Basin Hopping ในทุกกรณี ในมิติต่ำ (2D-4D) Dual Annealing มีประสิทธิภาพสูงกว่าอย่างมีนัยสำคัญ โดยเร็วกว่า 2.8-14.4 เท่า ซึ่งเป็นผลมาจากการใช้ Cauchy-Lorentz Distribution ที่มีหางหนา ทำให้สามารถกระโดดข้ามบริเวณ Local Minima ได้อย่างมีประสิทธิภาพ โดยไม่ต้องสำรวจทุกพื้นที่อย่างละเอียด การกระโดดระยะไกลนี้ช่วยให้ Algorithm หลีกเลียงการติดอยู่ใน Basins ที่ไม่ดี และเข้าถึง Basin ของ Global Minimum ได้เร็วขึ้น โดยเฉพาะในมิติ 2D ที่แสดงความได้เปรียบสูงสุด อย่างไรก็ตาม ในมิติสูง (5D-6D) ความแตกต่างด้านประสิทธิภาพลดลงเหลือเพียง 1.3 เท่า แสดงให้เห็นว่าเมื่อความซับซ้อนของปัญหาเพิ่มขึ้น พื้นที่ค้นหาขยายตัวแบบ Exponential และจำนวน Local Minima เพิ่มขึ้นอย่างมหาศาล ทำให้ทั้งสอง Algorithms มีประสิทธิภาพที่ใกล้เคียงกันมากขึ้น สิ่งนี้สะท้อนให้เห็นถึง "Curse Of Dimensionality" ที่ส่งผลกระทบต่อทุก Global Optimization Algorithm

รูปแบบ Acceptance Rate เผยให้เห็นลักษณะเฉพาะของแต่ละ Algorithm อย่างชัดเจน Dual Annealing แสดงพฤติกรรม "เลือกสรร" โดยมี Acceptance Rate ที่เพิ่มขึ้นตามมิติ จาก 1.4% ในมิติ 2D เป็น 83.1% ในมิติ 6D ในมิติต่ำ Algorithm ยอมรับเฉพาะจุดคุณภาพสูง ส่งผลให้ใช้การประเมิน Function อย่างมีประสิทธิภาพ เมื่อมิติเพิ่มขึ้น Temperature Schedule ที่ปรับตามจำนวนมิติทำให้อุณหภูมิลดต่ำลง ส่งผลให้ยอมรับการเคลื่อนที่ได้มากขึ้น สะท้อนความจำเป็นในการสำรวจพื้นที่กว้างขึ้นในปัญหามิติสูง ในทางตรงกันข้าม Basin Hopping รักษา Acceptance Rate ไว้ในช่วงแคบ (39.8-54.4%) อย่างสม่ำเสมอทุกมิติ สะท้อนกลไก Adaptive Step Size ที่ปรับขนาดก้าวอัตโนมัติ เพื่อรักษาเสถียรภาพ แนวทางนี้ให้ความมั่นคงและคาดการณ์พฤติกรรมได้ง่าย

ความแตกต่างนี้แสดงให้เห็นว่า Dual Annealing เหมาะสำหรับปัญหาที่ต้องการ ประสิทธิภาพสูงสุดโดยเฉพาะในมิติต่ำ ในขณะที่ Basin Hopping เหมาะสำหรับ สถานการณ์ที่ต้องการความเสถียรและคาดการณ์พฤติกรรมได้ง่าย

Reference

- Tsallis, C., & Stariolo, D. A. (1996). Generalized simulated annealing. *Physica A: Statistical Mechanics and Its Applications*, 233(3-4), 395-406. [https://doi.org/10.1016/S0378-4371\(96\)00271-3](https://doi.org/10.1016/S0378-4371(96)00271-3)
- Xiang, Y., Sun, D. Y., Fan, W., & Gong, X. G. (1997). Generalized simulated annealing algorithm and its application to the Thomson model. *Physics Letters A*, 233(3), 216-220. [https://doi.org/10.1016/S0375-9601\(97\)00474-X](https://doi.org/10.1016/S0375-9601(97)00474-X)
- Wales, D. J., & Doye, J. P. K. (1997). Global optimization by basin-hopping and the lowest energy structures of Lennard-Jones clusters containing up to 110 atoms. *The Journal of Physical Chemistry A*, 101(28), 5111-5116. <https://doi.org/10.1021/jp970984n>
- Xiang, Y., Gubian, S., Suomela, B., & Hoeng, J. (2013). Generalized simulated annealing for global optimization: The GenSA package. *The R Journal*, 5(1), 13-28. <https://doi.org/10.32614/RJ-2013-002>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261-272. <https://doi.org/10.1038/s41592-019-0686-2>